I'll explain pagination step by step, referencing your project structure and showing exactly how you've implemented it.

What is Pagination and Why We Need It

What is Pagination?

Pagination is the process of dividing large datasets into smaller, manageable chunks (pages) instead of loading all records at once.

Why We Need Pagination in Your Project

Performance Problems Without Pagination

Looking at your current EmployeesController.cs:

```
[HttpGet]
public async Task<IActionResult> GetEmployees()
{
    var empList = await _employeeService.GetAllEmployees(); // ❌
PROBLEM: Loads ALL employees
    return Ok(empList);
}
```

If you have 10,000 employees:

- **Memory Usage**: Loads all 10,000 records into server memory
- **Network Transfer**: Sends all 10,000 records to client
- **Response Time**: 2-5 seconds (slow user experience)
- **Database Load**: Full table scan every request

Benefits With Pagination

- **Memory Efficient**: Only 10-50 records loaded at once
- **Fast Response**: 50-200ms response time

- **Better UX**: Users see data quickly
- **Scalable**: Handles millions of records efficiently

Step-by-Step Pagination Implementation in Your Project

Your Current Project Structure Analysis

Looking at your current code, I can see you have:

- **Controller Layer**: EmployeesController.cs
- **Service Interface**: IEmployeeService.cs
- **Repository Layer**: EmployeeRepositories.cs
- **Entity**: EmpMaster.cs

Current Problem: Your GetEmployees() method loads ALL employees at once.

Step 1: Create Pagination Models

1.1 Create Pagination Parameters

Create `CleanArchCRUD.Applications/DTOs/PaginationParameters.cs`:

```
using System.ComponentModel.DataAnnotations;

namespace CleanArchCRUD.Applications.DTOs
{
    public class PaginationParameters
    {
        private const int MaxPageSize = 50;
        private int _pageSize = 10;

        [Range(1, int.MaxValue, ErrorMessage = "Page number must be
greater than 0")]
        public int PageNumber { get; set; } = 1;
```

```
        [Range(1, MaxPageSize, ErrorMessage = $"Page size must be
between 1 and {MaxPageSize}")]
        public int PageSize
        {
            get => _pageSize;
            set => _pageSize = (value > MaxPageSize) ? MaxPageSize :
value;
        }

        public string? SearchTerm { get; set; }
        public string? SortBy { get; set; }
        public bool SortDescending { get; set; } = false;
    }
}
```

1.2 Create Paged Response Model

Create `CleanArchCRUD.Applications/DTOs/PagedResponse.cs`:

```
namespace CleanArchCRUD.Applications.DTOs
{
    public class PagedResponse
    {
        public int CurrentPage { get; set; }
        public int PageSize { get; set; }
        public int TotalCount { get; set; }
        public int TotalPages { get; set; }
        public bool HasPreviousPage => CurrentPage > 1;
        public bool HasNextPage => CurrentPage < TotalPages;
    }

    public class PagedResponse<T> : PagedResponse where T : class
    {
        public IEnumerable<T> Data { get; set; } = new List<T>();
    }
}
```

Step 2: Update Repository Layer

2.1 Update Repository Interface

Modify CleanArchCRUD.Applications/Interfaces/IEmployeeRepository.cs:

```
using CleanArchCRUD.Applications.DTOs;
using CleanArchCRUD.Domain.Entities;

namespace CleanArchCRUD.Domain.IRepositories
{
    public interface IEmployeeRepository
    {
        Task<IEnumerable<EmpMaster?>> GetEmployeeList();
        Task<EmpMaster?> GetEmployeeByempId(int empId);
        Task AddEmployee(EmpMaster empMaster);
        Task UpdateEmployee(EmpMaster empMaster);
        Task DeleteEmployee(EmpMaster empMaster);
        Task SaveChangesAsync();

        // NEW: Pagination method
        Task<PagedResponse<EmpMaster>>
GetEmployeesPagedAsync(PaginationParameters parameters);
    }
}
```

2.2 Update Repository Implementation

Modify CleanArchCRUD.Infrastructures/Repositories/EmployeeRepositories.cs:

```
using CleanArchCRUD.Applications.DTOs;
using CleanArchCRUD.Domain.Entities;
using CleanArchCRUD.Domain.IRepositories;
using Microsoft.EntityFrameworkCore;

namespace CleanArchCRUD.Applications.Repositories
{
    public class EmployeeRepositories : IEmployeeRepository
```

```
{
    private readonly employeeDbContext _empDbContext;

    public EmployeeRepositories(employeeDbContext empDbContext)
    {
        _empDbContext = empDbContext;
    }

    // Existing methods remain the same...
    public async Task<IEnumerable<EmpMaster?>> GetEmployeeList()
    {
        return await _empDbContext.EmpMasters.ToListAsync();
    }

    public async Task<EmpMaster?> GetEmployeeByempId(int empId)
    {
        return await _empDbContext.EmpMasters.FindAsync(empId);
    }

    public async Task AddEmployee(EmpMaster empMaster)
    {
        await _empDbContext.EmpMasters.AddAsync(empMaster);
    }

    public async Task UpdateEmployee(EmpMaster empMaster)
    {
        _empDbContext.EmpMasters.Update(empMaster);
        await Task.CompletedTask;
    }

    public async Task DeleteEmployee(EmpMaster empMaster)
    {
        _empDbContext.EmpMasters.Remove(empMaster);
        await Task.CompletedTask;
    }

    public async Task SaveChangesAsync()
    {
        await _empDbContext.SaveChangesAsync();
```

```
        }

        // NEW: Pagination implementation
        public async Task<PagedResponse<EmpMaster>>
GetEmployeesPagedAsync(PaginationParameters parameters)
        {
            var query = _empDbContext.EmpMasters.AsQueryable();

            // Apply search filter
            if (!string.IsNullOrWhiteSpace(parameters.SearchTerm))
            {
                query = query.Where(e =>
                    e.EmpName!.Contains(parameters.SearchTerm) ||
                    e.empemail!.Contains(parameters.SearchTerm));
            }

            // Apply sorting
            if (!string.IsNullOrWhiteSpace(parameters.SortBy))
            {
                switch (parameters.SortBy.ToLower())
                {
                    case "name":
                        query = parameters.SortDescending
                            ? query.OrderByDescending(e => e.EmpName)
                            : query.OrderBy(e => e.EmpName);
                        break;
                    case "email":
                        query = parameters.SortDescending
                            ? query.OrderByDescending(e => e.empemail)
                            : query.OrderBy(e => e.empemail);
                        break;
                    default:
                        query = parameters.SortDescending
                            ? query.OrderByDescending(e => e.EmpId)
                            : query.OrderBy(e => e.EmpId);
                        break;
                }
            }
            else
```

```
            {
                query = query.OrderBy(e => e.EmpId);
            }

            // Get total count
            var totalCount = await query.CountAsync();

            // Apply pagination
            var items = await query
                .Skip((parameters.PageNumber - 1) *
parameters.PageSize)
                .Take(parameters.PageSize)
                .ToListAsync();

            return new PagedResponse<EmpMaster>
            {
                Data = items,
                CurrentPage = parameters.PageNumber,
                PageSize = parameters.PageSize,
                TotalCount = totalCount,
                TotalPages = (int)Math.Ceiling(totalCount /
(double)parameters.PageSize)
            };
        }
    }
}
```

Step 3: Update Service Layer

3.1 Update Service Interface

Modify CleanArchCRUD.Applications/Interfaces/IEmployeeService.cs:

```
using CleanArchCRUD.Applications.DTOs;
using CleanArchCRUD.Domain.Entities;

namespace CleanArchCRUD.Domain.Interfaces
{
```

```
    public interface IEmployeeService
    {
        Task<IEnumerable<EmpMaster?>> GetAllEmployees();
        Task<EmpMaster> GetEmpById(int empId);
        Task<EmpMaster> CreateEmp(EmpMasterDTO empMasterDTO);
        Task UpdateEmp(EmpMasterDTO empMasterDTO);
        Task DeleteTask(int empId);

        // NEW: Pagination method
        Task<PagedResponse<EmpMaster>>
GetEmployeesPagedAsync(PaginationParameters parameters);
    }
}
```

3.2 Update Service Implementation

Modify CleanArchCRUD.Infrastructures/Services/EmployeeService.cs:

```
using CleanArchCRUD.Applications.DTOs;
using CleanArchCRUD.Domain.Entities;
using CleanArchCRUD.Domain.Interfaces;
using CleanArchCRUD.Domain.IRepositories;

namespace CleanArchCRUD.Infrastructures.Services
{
    public class EmployeeService : IEmployeeService
    {
        private readonly IEmployeeRepository _employeeRepository;

        public EmployeeService(IEmployeeRepository employeeRepository)
        {
            _employeeRepository = employeeRepository;
        }

        // Existing methods remain the same...
        public async Task<IEnumerable<EmpMaster?>> GetAllEmployees()
        {
           return await _employeeRepository.GetEmployeeList();
```

```
        }

        public async Task<EmpMaster> GetEmpById(int empId)
        {
            var task = await
_employeeRepository.GetEmployeeByempId(empId);
            if(task==null)
            {
                throw new Exception("Employee not found");
            }
            await _employeeRepository.SaveChangesAsync();
            return task;
        }

        public async Task<EmpMaster> CreateEmp(EmpMasterDTO
empMasterDTO)
        {
            var task = new EmpMaster
            {
                EmpName = empMasterDTO.EmpName,
                empemail = empMasterDTO.empemail,
            };
            await _employeeRepository.AddEmployee(task);
            await _employeeRepository.SaveChangesAsync();
            return task;
        }

        public async Task UpdateEmp(EmpMasterDTO empMasterDTO)
        {
            var task = await
_employeeRepository.GetEmployeeByempId(empMasterDTO.EmpId);
            if (task == null)
            {
                throw new Exception("Employee not found");
            }
            task.EmpName = empMasterDTO.EmpName;
            task.empemail = empMasterDTO.empemail;

            await _employeeRepository.UpdateEmployee(task);
```

```
            await _employeeRepository.SaveChangesAsync();
        }

        public async Task DeleteTask(int empId)
        {
            var task = await
_employeeRepository.GetEmployeeByempId(empId);
            if (task == null)
            {
                throw new Exception("Employee Not Found");
            }
            await _employeeRepository.DeleteEmployee(task);
            await _employeeRepository.SaveChangesAsync();
        }

        // NEW: Pagination method
        public async Task<PagedResponse<EmpMaster>>
GetEmployeesPagedAsync(PaginationParameters parameters)
        {
            return await
_employeeRepository.GetEmployeesPagedAsync(parameters);
        }
    }
}
```

Step 4: Update Controller Layer

Modify CleanArchCRUD.API/Controllers/EmployeesController.cs:

```
using CleanArchCRUD.Applications.DTOs;
using CleanArchCRUD.Domain.Interfaces;
using Microsoft.AspNetCore.Mvc;

namespace CleanArchCRUD.API.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    public class EmployeesController : ControllerBase
```

```
    {
        private readonly IEmployeeService _employeeService;

        public EmployeesController(IEmployeeService employeeService)
        {
            _employeeService = employeeService;
        }

        // UPDATED: Now supports pagination
        [HttpGet]
        public async Task<IActionResult> GetEmployees([FromQuery]
PaginationParameters parameters)
        {
            var pagedEmployees = await
_employeeService.GetEmployeesPagedAsync(parameters);
            return Ok(pagedEmployees);
        }

        // OPTIONAL: Keep original method for backward compatibility
        [HttpGet("all")]
        public async Task<IActionResult> GetAllEmployees()
        {
            var empList = await _employeeService.GetAllEmployees();
            return Ok(empList);
        }

        [HttpPost]
        public async Task<IActionResult>
Createemployee([FromBody]EmpMasterDTO empMasterDTO)
        {
            if(string.IsNullOrEmpty(empMasterDTO.EmpName))
            {
                return BadRequest("Employee Name is Required");
            }
            var task = await _employeeService.CreateEmp(empMasterDTO);
            return Ok(task);
        }

        [HttpPut]
```

```
        public async Task<IActionResult> updateEmpl([FromBody]
EmpMasterDTO empMasterDTO)
        {
            if(string.IsNullOrEmpty(empMasterDTO.EmpName))
            {
                return BadRequest("Employee Name is Required");
            }
            await _employeeService.UpdateEmp(empMasterDTO);
            return Ok(new { Message="Task Updated successfully !!" });
        }

        [HttpDelete]
        public async Task<IActionResult> deleteEmp(int empId)
        {
            if(empId == null)
            {
                return NotFound("Employee Id Not Found");
            }
            await _employeeService.DeleteTask(empId);
            return Ok(new {  Message="Task Deleted
successfully !!!" });
        }
    }
}
```

How Pagination Works in Each Layer of Your Project

Data Flow Through Your Clean Architecture

Layer 1: Controller (API Layer)

```
[HttpGet]
public async Task<IActionResult> GetEmployees([FromQuery]
PaginationParameters parameters)
{
```

```
    // Receives URL:
/api/employees?pageNumber=2&pageSize=10&searchTerm=john&sortBy=name
    var pagedEmployees = await
_employeeService.GetEmployeesPagedAsync(parameters);
    return Ok(pagedEmployees);
}
```

What happens here:

- **Input**: Receives query parameters from URL
- **Validation**: `[FromQuery]` automatically binds and validates parameters
- **Delegation**: Passes parameters to service layer
- **Response**: Returns structured JSON with pagination metadata

Layer 2: Service (Business Logic Layer)

```
public async Task<PagedResponse<EmpMaster>>
GetEmployeesPagedAsync(PaginationParameters parameters)
{
    // Could add business logic here (validation, business rules,
etc.)
    return await
_employeeRepository.GetEmployeesPagedAsync(parameters);
}
```

What happens here:

- **Business Logic**: Could add validation, business rules, logging
- **Delegation**: Passes to repository layer
- **Abstraction**: Hides data access details from controller

Layer 3: Repository (Data Access Layer)

```
public async Task<PagedResponse<EmpMaster>>
GetEmployeesPagedAsync(PaginationParameters parameters)
{
    var query = _empDbContext.EmpMasters.AsQueryable();
```

```
    // 1. Apply search filter
    if (!string.IsNullOrWhiteSpace(parameters.SearchTerm))
    {
        query = query.Where(e =>
            e.EmpName!.Contains(parameters.SearchTerm) ||
            e.empemail!.Contains(parameters.SearchTerm));
    }

    // 2. Apply sorting
    query = query.OrderBy(e => e.EmpName);

    // 3. Get total count for pagination metadata
    var totalCount = await query.CountAsync();

    // 4. Apply pagination (SKIP + TAKE)
    var items = await query
        .Skip((parameters.PageNumber - 1) * parameters.PageSize)
        .Take(parameters.PageSize)
        .ToListAsync();

    // 5. Build response
    return new PagedResponse<EmpMaster>
    {
        Data = items,
        CurrentPage = parameters.PageNumber,
        PageSize = parameters.PageSize,
        TotalCount = totalCount,
        TotalPages = (int)Math.Ceiling(totalCount /
(double)parameters.PageSize)
    };
}
```

What happens here:

- **Query Building**: Creates efficient SQL query with WHERE, ORDER BY
- **Search**: Applies search filters to name and email
- **Sorting**: Orders results by specified field
- **Counting**: Gets total record count for pagination metadata

- **Pagination**: Uses `Skip()` and `Take()` for efficient paging
- **Response**: Builds structured response with metadata

Practical Usage Examples

API Usage Examples

Basic Pagination

```
GET /api/employees?pageNumber=1&pageSize=10
```

Response:

```
{
  "data": [
    {"empId": 1, "empName": "John Doe", "empemail":
"john@example.com"},
    {"empId": 2, "empName": "Jane Smith", "empemail":
"jane@example.com"}
  ],
  "currentPage": 1,
  "pageSize": 10,
  "totalCount": 25,
  "totalPages": 3,
  "hasPreviousPage": false,
  "hasNextPage": true
}
```

With Search

```
GET /api/employees?pageNumber=1&pageSize=5&searchTerm=john
```

With Sorting

```
GET
/api/employees?pageNumber=1&pageSize=10&sortBy=name&sortDescending=tru
e
```

Combined Example

```
GET
/api/employees?pageNumber=2&pageSize=5&searchTerm=@example.com&sortBy=
email
```

Database Query Generated

Entity Framework converts your C# code to efficient SQL:

```
-- For pageNumber=2&pageSize=10&searchTerm=john&sortBy=name
SELECT [e].[EmpId], [e].[EmpName], [e].[empemail], [e].[empPassword]
FROM [EmpMasters] AS [e]
WHERE ([e].[EmpName] LIKE N'%john%') OR ([e].[empemail] LIKE
N'%john%')
ORDER BY [e].[EmpName]
OFFSET 10 ROWS FETCH NEXT 10 ROWS ONLY

-- Separate query for total count
SELECT COUNT(*)
FROM [EmpMasters] AS [e]
WHERE ([e].[EmpName] LIKE N'%john%') OR ([e].[empemail] LIKE
N'%john%')
```

Performance Comparison

Before Pagination (Your Current Code)

```
[HttpGet]
public async Task<IActionResult> GetEmployees()
{
    var empList = await _employeeService.GetAllEmployees(); // ❌
Loads ALL records
    return Ok(empList);
}
```

- **Memory**: Loads 10,000+ records
- **Network**: Sends all records
- **Response Time**: 2-5 seconds

After Pagination

```
[HttpGet]
public async Task<IActionResult> GetEmployees([FromQuery]
PaginationParameters parameters)
{
    var pagedEmployees = await
_employeeService.GetEmployeesPagedAsync(parameters); // ✅ Loads 10
records
    return Ok(pagedEmployees);
}
```

- **Memory**: Loads only 10 records
- **Network**: Sends only 10 records
- **Response Time**: 50-200ms

Frontend Integration Example

JavaScript/React Component

```
async function fetchEmployees(page = 1, pageSize = 10, searchTerm =
'') {
  const response = await fetch(

`/api/employees?pageNumber=${page}&pageSize=${pageSize}&searchTerm=${s
earchTerm}`
  );
  const data = await response.json();

  return {
    employees: data.data,
    currentPage: data.currentPage,
    totalPages: data.totalPages,
    hasNextPage: data.hasNextPage,
    hasPreviousPage: data.hasPreviousPage
  };
}

// Usage
const result = await fetchEmployees(2, 10, 'john');
console.log(result.employees); // Array of 10 employees
console.log(result.currentPage); // 2
console.log(result.totalPages); // 5
```

Summary: Complete Pagination Implementation

What is Pagination?

Pagination divides large datasets into smaller, manageable chunks (pages) instead of loading all records at once.

Why You Need It

Your current GetEmployees() method loads ALL employees, causing:

- **Memory Issues**: Loads thousands of records into server memory
- **Slow Response**: 2-5 seconds for large datasets
- **Poor UX**: Users wait long for data to load
- **Scalability Problems**: Doesn't handle growth well

How You Achieved It - Step by Step

Step 1: Created Pagination Models

- `PaginationParameters.cs`: Handles page number, size, search, sorting
- `PagedResponse.cs`: Contains data + pagination metadata

Step 2: Updated Repository Layer

- Added `GetEmployeesPagedAsync()` method
- Implemented efficient SQL with `Skip()` and `Take()`
- Added search and sorting capabilities

Step 3: Updated Service Layer

- Added pagination method to interface and implementation
- Maintains clean separation of concerns

Step 4: Updated Controller Layer

- Modified GetEmployees() to accept query parameters
- Returns structured response with pagination metadata

Performance Results

- **Before**: 2-5 seconds, loads all records
- **After**: 50-200ms, loads only 10-50 records
- **Memory Usage**: Reduced by 90%+

- **Network Transfer**: Reduced by 90%+

Usage Examples

```
GET
/api/employees?pageNumber=2&pageSize=10&searchTerm=john&sortBy=name
```

Your Clean Architecture CRUD application now has efficient, scalable pagination that handles large datasets effectively while providing excellent user experience.

Interview Question

I'll provide you with complete pagination code that you can write in an interview. This is a comprehensive, production-ready implementation.

Complete Pagination Code for Interview

1. Pagination Parameters Model

```
using System.ComponentModel.DataAnnotations;

namespace Application.DTOs
{
    public class PaginationParameters
    {
        private const int MaxPageSize = 50;
        private int _pageSize = 10;

        [Range(1, int.MaxValue, ErrorMessage = "Page number must be
greater than 0")]
        public int PageNumber { get; set; } = 1;

        [Range(1, MaxPageSize, ErrorMessage = $"Page size must be
```

```
between 1 and {MaxPageSize}")]
        public int PageSize
        {
            get => _pageSize;
            set => _pageSize = (value > MaxPageSize) ? MaxPageSize :
value;
        }

        public string? SearchTerm { get; set; }
        public string? SortBy { get; set; }
        public bool SortDescending { get; set; } = false;
    }
}
```

2. Paged Response Model

```
namespace Application.DTOs
{
    public class PagedResponse
    {
        public int CurrentPage { get; set; }
        public int PageSize { get; set; }
        public int TotalCount { get; set; }
        public int TotalPages { get; set; }
        public bool HasPreviousPage => CurrentPage > 1;
        public bool HasNextPage => CurrentPage < TotalPages;
        public int FirstRowOnPage => (CurrentPage - 1) * PageSize + 1;
        public int LastRowOnPage => Math.Min(CurrentPage * PageSize,
TotalCount);
    }

    public class PagedResponse<T> : PagedResponse where T : class
    {
        public IEnumerable<T> Data { get; set; } = new List<T>();
    }
}
```

3. Repository Interface

```
using Application.DTOs;
using Domain.Entities;

namespace Domain.Interfaces
{
    public interface IEmployeeRepository
    {
        Task<IEnumerable<Employee>> GetAllEmployeesAsync();
        Task<Employee?> GetEmployeeByIdAsync(int id);
        Task<Employee> CreateEmployeeAsync(Employee employee);
        Task UpdateEmployeeAsync(Employee employee);
        Task DeleteEmployeeAsync(Employee employee);
        Task<int> SaveChangesAsync();

        // Pagination method
        Task<PagedResponse<Employee>>
GetEmployeesPagedAsync(PaginationParameters parameters);
    }
}
```

4. Repository Implementation

```
using Application.DTOs;
using Domain.Entities;
using Domain.Interfaces;
using Microsoft.EntityFrameworkCore;

namespace Infrastructure.Repositories
{
    public class EmployeeRepository : IEmployeeRepository
    {
        private readonly ApplicationDbContext _context;

        public EmployeeRepository(ApplicationDbContext context)
        {
```

```
            _context = context;
        }

        public async Task<IEnumerable<Employee>>
GetAllEmployeesAsync()
        {
            return await _context.Employees.ToListAsync();
        }

        public async Task<Employee?> GetEmployeeByIdAsync(int id)
        {
            return await _context.Employees.FindAsync(id);
        }

        public async Task<Employee> CreateEmployeeAsync(Employee
employee)
        {
            await _context.Employees.AddAsync(employee);
            await _context.SaveChangesAsync();
            return employee;
        }

        public async Task UpdateEmployeeAsync(Employee employee)
        {
            _context.Employees.Update(employee);
            await _context.SaveChangesAsync();
        }

        public async Task DeleteEmployeeAsync(Employee employee)
        {
            _context.Employees.Remove(employee);
            await _context.SaveChangesAsync();
        }

        public async Task<int> SaveChangesAsync()
        {
            return await _context.SaveChangesAsync();
        }
```

```
        // Pagination implementation
        public async Task<PagedResponse<Employee>>
GetEmployeesPagedAsync(PaginationParameters parameters)
        {
            var query = _context.Employees.AsQueryable();

            // Apply search filter
            if (!string.IsNullOrWhiteSpace(parameters.SearchTerm))
            {
                query = query.Where(e =>
                    e.FirstName.Contains(parameters.SearchTerm) ||
                    e.LastName.Contains(parameters.SearchTerm) ||
                    e.Email.Contains(parameters.SearchTerm));
            }

            // Apply sorting
            if (!string.IsNullOrWhiteSpace(parameters.SortBy))
            {
                switch (parameters.SortBy.ToLower())
                {
                    case "name":
                        query = parameters.SortDescending
                            ? query.OrderByDescending(e =>
e.FirstName).ThenByDescending(e => e.LastName)
                            : query.OrderBy(e => e.FirstName).ThenBy(e
=> e.LastName);
                        break;
                    case "email":
                        query = parameters.SortDescending
                            ? query.OrderByDescending(e => e.Email)
                            : query.OrderBy(e => e.Email);
                        break;
                    case "salary":
                        query = parameters.SortDescending
                            ? query.OrderByDescending(e => e.Salary)
                            : query.OrderBy(e => e.Salary);
                        break;
                    default:
                        query = parameters.SortDescending
```

```
                            ? query.OrderByDescending(e => e.Id)
                            : query.OrderBy(e => e.Id);
                        break;
                }
            }
            else
            {
                // Default sort by Id
                query = query.OrderBy(e => e.Id);
            }

            // Get total count
            var totalCount = await query.CountAsync();

            // Apply pagination
            var items = await query
                .Skip((parameters.PageNumber - 1) *
parameters.PageSize)
                .Take(parameters.PageSize)
                .ToListAsync();

            return new PagedResponse<Employee>
            {
                Data = items,
                CurrentPage = parameters.PageNumber,
                PageSize = parameters.PageSize,
                TotalCount = totalCount,
                TotalPages = (int)Math.Ceiling(totalCount /
(double)parameters.PageSize)
            };
        }
    }
}
```

5. Service Interface

```
using Application.DTOs;
using Domain.Entities;
```

```
namespace Domain.Interfaces
{
    public interface IEmployeeService
    {
        Task<IEnumerable<Employee>> GetAllEmployeesAsync();
        Task<Employee?> GetEmployeeByIdAsync(int id);
        Task<Employee> CreateEmployeeAsync(CreateEmployeeDto
employeeDto);
        Task UpdateEmployeeAsync(UpdateEmployeeDto employeeDto);
        Task DeleteEmployeeAsync(int id);

        // Pagination method
        Task<PagedResponse<Employee>>
GetEmployeesPagedAsync(PaginationParameters parameters);
    }
}
```

6. Service Implementation

```
using Application.DTOs;
using Domain.Entities;
using Domain.Interfaces;

namespace Application.Services
{
    public class EmployeeService : IEmployeeService
    {
        private readonly IEmployeeRepository _employeeRepository;

        public EmployeeService(IEmployeeRepository employeeRepository)
        {
            _employeeRepository = employeeRepository;
        }

        public async Task<IEnumerable<Employee>>
GetAllEmployeesAsync()
        {
```

```
            return await _employeeRepository.GetAllEmployeesAsync();
        }

        public async Task<Employee?> GetEmployeeByIdAsync(int id)
        {
            return await _employeeRepository.GetEmployeeByIdAsync(id);
        }

        public async Task<Employee>
CreateEmployeeAsync(CreateEmployeeDto employeeDto)
        {
            var employee = new Employee
            {
                FirstName = employeeDto.FirstName,
                LastName = employeeDto.LastName,
                Email = employeeDto.Email,
                Salary = employeeDto.Salary,
                Department = employeeDto.Department,
                CreatedDate = DateTime.UtcNow
            };

            return await
_employeeRepository.CreateEmployeeAsync(employee);
        }

        public async Task UpdateEmployeeAsync(UpdateEmployeeDto
employeeDto)
        {
            var existingEmployee = await
_employeeRepository.GetEmployeeByIdAsync(employeeDto.Id);
            if (existingEmployee == null)
                throw new KeyNotFoundException($"Employee with ID
{employeeDto.Id} not found");

            existingEmployee.FirstName = employeeDto.FirstName;
            existingEmployee.LastName = employeeDto.LastName;
            existingEmployee.Email = employeeDto.Email;
            existingEmployee.Salary = employeeDto.Salary;
            existingEmployee.Department = employeeDto.Department;
```

```
            existingEmployee.ModifiedDate = DateTime.UtcNow;

            await
_employeeRepository.UpdateEmployeeAsync(existingEmployee);
        }

        public async Task DeleteEmployeeAsync(int id)
        {
            var employee = await
_employeeRepository.GetEmployeeByIdAsync(id);
            if (employee == null)
                throw new KeyNotFoundException($"Employee with ID {id}
not found");

            await _employeeRepository.DeleteEmployeeAsync(employee);
        }

        // Pagination implementation
        public async Task<PagedResponse<Employee>>
GetEmployeesPagedAsync(PaginationParameters parameters)
        {
            // Business logic validation
            if (parameters.PageNumber < 1)
                throw new ArgumentException("Page number must be
greater than 0");

            if (parameters.PageSize < 1 || parameters.PageSize > 50)
                throw new ArgumentException("Page size must be between
1 and 50");

            return await
_employeeRepository.GetEmployeesPagedAsync(parameters);
        }
    }
}
```

7. Controller with Pagination Endpoint

```
using Application.DTOs;
using Domain.Interfaces;
using Microsoft.AspNetCore.Mvc;

namespace API.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    public class EmployeesController : ControllerBase
    {
        private readonly IEmployeeService _employeeService;

        public EmployeesController(IEmployeeService employeeService)
        {
            _employeeService = employeeService;
        }

        // Pagination endpoint
        [HttpGet]
        public async Task<IActionResult> GetEmployees([FromQuery]
PaginationParameters parameters)
        {
            try
            {
                var pagedEmployees = await
_employeeService.GetEmployeesPagedAsync(parameters);
                return Ok(pagedEmployees);
            }
            catch (ArgumentException ex)
            {
                return BadRequest(ex.Message);
            }
        }

        // Get all employees (non-paginated for comparison)
        [HttpGet("all")]
        public async Task<IActionResult> GetAllEmployees()
```

```
        {
            var employees = await
_employeeService.GetAllEmployeesAsync();
            return Ok(employees);
        }

        [HttpGet("{id}")]
        public async Task<IActionResult> GetEmployee(int id)
        {
            var employee = await
_employeeService.GetEmployeeByIdAsync(id);
            if (employee == null)
                return NotFound($"Employee with ID {id} not found");

            return Ok(employee);
        }

        [HttpPost]
        public async Task<IActionResult> CreateEmployee([FromBody]
CreateEmployeeDto employeeDto)
        {
            if (!ModelState.IsValid)
                return BadRequest(ModelState);

            try
            {
                var employee = await
_employeeService.CreateEmployeeAsync(employeeDto);
                return CreatedAtAction(nameof(GetEmployee), new { id =
employee.Id }, employee);
            }
            catch (Exception ex)
            {
                return StatusCode(500, "An error occurred while
creating the employee");
            }
        }

        [HttpPut("{id}")]
```

```
        public async Task<IActionResult> UpdateEmployee(int id,
[FromBody] UpdateEmployeeDto employeeDto)
        {
            if (!ModelState.IsValid)
                return BadRequest(ModelState);

            if (id != employeeDto.Id)
                return BadRequest("ID mismatch");

            try
            {
                await
_employeeService.UpdateEmployeeAsync(employeeDto);
                return NoContent();
            }
            catch (KeyNotFoundException ex)
            {
                return NotFound(ex.Message);
            }
            catch (Exception ex)
            {
                return StatusCode(500, "An error occurred while
updating the employee");
            }
        }

        [HttpDelete("{id}")]
        public async Task<IActionResult> DeleteEmployee(int id)
        {
            try
            {
                await _employeeService.DeleteEmployeeAsync(id);
                return NoContent();
            }
            catch (KeyNotFoundException ex)
            {
                return NotFound(ex.Message);
            }
            catch (Exception ex)
```

```
            {
                return StatusCode(500, "An error occurred while
deleting the employee");
            }
        }
    }
}
```

8. Supporting Models and DTOs

```
// Employee Entity
namespace Domain.Entities
{
    public class Employee
    {
        public int Id { get; set; }
        public string FirstName { get; set; } = string.Empty;
        public string LastName { get; set; } = string.Empty;
        public string Email { get; set; } = string.Empty;
        public decimal Salary { get; set; }
        public string Department { get; set; } = string.Empty;
        public DateTime CreatedDate { get; set; }
        public DateTime? ModifiedDate { get; set; }
    }
}

// Create Employee DTO
namespace Application.DTOs
{
    public class CreateEmployeeDto
    {
        [Required]
        [StringLength(50)]
        public string FirstName { get; set; } = string.Empty;

        [Required]
        [StringLength(50)]
        public string LastName { get; set; } = string.Empty;
```

```
        [Required]
        [EmailAddress]
        public string Email { get; set; } = string.Empty;

        [Range(0, double.MaxValue)]
        public decimal Salary { get; set; }

        [Required]
        [StringLength(100)]
        public string Department { get; set; } = string.Empty;
    }

    // Update Employee DTO
    public class UpdateEmployeeDto
    {
        public int Id { get; set; }

        [Required]
        [StringLength(50)]
        public string FirstName { get; set; } = string.Empty;

        [Required]
        [StringLength(50)]
        public string LastName { get; set; } = string.Empty;

        [Required]
        [EmailAddress]
        public string Email { get; set; } = string.Empty;

        [Range(0, double.MaxValue)]
        public decimal Salary { get; set; }

        [Required]
        [StringLength(100)]
        public string Department { get; set; } = string.Empty;
    }
}
```

```
// DbContext
using Domain.Entities;
using Microsoft.EntityFrameworkCore;

namespace Infrastructure.Data
{
    public class ApplicationDbContext : DbContext
    {
        public
ApplicationDbContext(DbContextOptions<ApplicationDbContext> options) :
base(options)
        {
        }

        public DbSet<Employee> Employees { get; set; }

        protected override void OnModelCreating(ModelBuilder
modelBuilder)
        {
            modelBuilder.Entity<Employee>(entity =>
            {
                entity.HasKey(e => e.Id);
                entity.Property(e =>
e.FirstName).IsRequired().HasMaxLength(50);
                entity.Property(e =>
e.LastName).IsRequired().HasMaxLength(50);
                entity.Property(e =>
e.Email).IsRequired().HasMaxLength(100);
                entity.Property(e =>
e.Department).IsRequired().HasMaxLength(100);
                entity.HasIndex(e => e.Email).IsUnique();
            });
        }
    }
}
```

9. Usage Examples and API Testing

```
// Example API calls:

// Basic pagination
GET /api/employees?pageNumber=1&pageSize=10

// With search
GET /api/employees?pageNumber=1&pageSize=10&searchTerm=john

// With sorting
GET
/api/employees?pageNumber=1&pageSize=10&sortBy=name&sortDescending=tru
e

// Combined
GET
/api/employees?pageNumber=2&pageSize=5&searchTerm=@example.com&sortBy=
email

// Response format:
{
  "data": [
    {
      "id": 1,
      "firstName": "John",
      "lastName": "Doe",
      "email": "john@example.com",
      "salary": 50000,
      "department": "IT",
      "createdDate": "2024-01-01T00:00:00Z",
      "modifiedDate": null
    }
  ],
  "currentPage": 1,
  "pageSize": 10,
  "totalCount": 25,
  "totalPages": 3,
  "hasPreviousPage": false,
```

```
  "hasNextPage": true,
  "firstRowOnPage": 1,
  "lastRowOnPage": 10
}
```

10. Dependency Injection Setup

```
// In Program.cs
builder.Services.AddScoped<IEmployeeRepository, EmployeeRepository>();
builder.Services.AddScoped<IEmployeeService, EmployeeService>();
builder.Services.AddDbContext<ApplicationDbContext>(options =>

options.UseSqlServer(builder.Configuration.GetConnectionString("Defaul
tConnection")));
```

Key Interview Points to Mention

1. **Performance Benefits**: Only loads required records, reduces memory usage
2. **Database Efficiency**: Uses SKIP and TAKE in SQL for efficient pagination
3. **Search & Sort**: Dynamic query building with LINQ
4. **Validation**: Input validation and error handling
5. **Clean Architecture**: Proper separation of concerns across layers
6. **Metadata**: Provides pagination information for frontend
7. **Scalability**: Handles large datasets efficiently

This complete pagination implementation demonstrates clean architecture principles, efficient database operations, and comprehensive error handling - perfect for an interview scenario.